

The Integrated OMI Relation Pipeline

Scope, Integrity, Time, Incidence, Projection, Gauge, and Runtime Resolution

Status

Canonical implementation model.

This document explains how the following structures operate together:

```
FS / GS / RS / US
scope and gauge position

LOGOS / NOMOS / PATHOS
local Hamming integrity relations

LL / MM / NN
temporal Delta3 position

Tetragrammatron Relation Governor
relation closure and fold validation

Metatron Incidence Scribe
gauge incidence, polarity, and place-value inscription

Gnomonic Projection Azimuth
observer-relative orientation

Omicron
bounded low gauge

Omicron
whole-byte circular runtime resolver

Receipt
accepted-state record
```

The integrated narrative is:

```
FS/GS/RS/US establish where the relation is scoped.
```

LOGOS/NOMOS/PATHOS determine whether that scoped relation has survived carriage coherently.

LL/MM/NN place the coherent or incoherent relation inside a previous/present/forward temporal window.

Tetragrammatron determines whether the relation closes.

Metatron records how the relation crossed the gauge field.

The Gnomonic Projection Azimuth orients the checked temporal relation for observer-relative inspection.

Omicron supplies the bounded sign-and-place ruler.

Omnicon coordinates the complete low/high runtime.

Receipt records acceptance without confusing validation, inscription, or projection.

1. The Primary Separation

The system must preserve five independent questions.

Scope:

Where does the relation belong?

Integrity:

Did its local representation survive correctly?

Time:

Is it previous, present, or forward?

Incidence:

Through which gauge and polarity did it pass?

Projection:

How does it appear from this observer-relative orientation?

These questions correspond to different structures:

Scope

→ FS / GS / RS / US

Integrity

→ LOGOS / NOMOS / PATHOS

Time

→ LL / MM / NN

Incidence

→ Metatron Incidence Scribe

Projection

→ Gnomonic Projection Azimuth

A failure in one layer must not be silently reinterpreted as a failure in another.

For example:

a Hamming syndrome

does not determine temporal position

a temporal position

does not imply semantic acceptance

an azimuth

does not validate a relation

a projected image

does not modify the underlying relation

2. Omicron: The Bounded Low Gauge

Omicron owns the bounded low ruler:

0x00..0x7F

This provides 128 positions divided into four 32-position character sectors:

0x00..0x1F

0x20..0x3F

```
0x40..0x5F
0x60..0x7F
```

A low position decomposes as:

```
sector = byte >> 5
local  = byte & 0x1F
```

where:

```
sector ∈ {0,1,2,3}
local  ∈ {0..31}
```

Omicron determines:

```
which low sign is reachable
which low character sector is active
which local position is selected
whether the route remains hidden or becomes readable
which outer hexadecimal place-value lane is coupled to the sign
```

Omicron is therefore the bounded sign-and-place gauge.

It is not the complete runtime.

3. The Hidden-to-Readable Tangent

The low control region terminates at:

```
0x1F = US
```

The readable affine region begins at:

```
0x20 = SP
```

The transition:

```
0x1F  
→  
0x20
```

is the hidden-to-readable tangent.

The F-column branch is:

```
0x1F  US  
0x2F  /  
0x3F  ?  
0x4F  0  
0x5F  _  
0x6F  o
```

This produces:

```
US/  
→  
?0_o
```

and allows the folded root:

```
omi---imo
```

to unfold as:

```
omi---imo?0_o
```

The important implementation rule is:

```
reachability comes from the active gauge position
```

A parser must not simulate this with an unrelated Boolean such as:

```
dot_allowed = true
```

The symbol is legal because it exists in the active reachable ring, not because a permission bit was toggled.

4. Omnicron: The Complete Runtime Resolver

Omicron owns the full byte field as four circular regions:

```
0x00..0x1F
LOW CONTROL PLANE
32 positions

0x20..0x7F
LOW AFFINE SPACE
96 positions

0x80..0x9F
HIGH PROJECTIVE CONTROL PLANE
32 positions

0xA0..0xFF
HIGH PROJECTIVE SPACE
96 positions
```

The total is:

```
32 + 96 + 32 + 96 = 256
```

Omicron determines:

```
current plane

current circular cursor

low/high polarity

control/space classification

affine/projective correspondence

runtime cadence

permitted tangent transition
```

candidate routing

The relation between the two names is:

Omicron defines the bounded low gauge.

Omicron resolves that gauge
against the complete low/high byte field.

5. FS, GS, RS, and US as Scope Coordinates

The four scope-bearing positions are:

FS
GS
RS
US

Their canonical meanings are:

FS
file or frame scope

GS
group or graph scope

RS
record or relation scope

US
unit or symbol scope

They answer:

Which containment boundaries are visible
for this relation?

A compact scope value may be stored as four bits:

```
bit 0 = FS  
bit 1 = GS  
bit 2 = RS  
bit 3 = US
```

For example:

```
scope4 = 0b1101
```

means:

```
FS present  
GS absent  
RS present  
US present
```

These four bits are the data positions of the Hamming epistemic cell.

6. The Hexadecimal Gauge Ladder

The same four gauges also have visible place-value masks:

```
FS = 0x0001  
GS = 0x0010  
RS = 0x0100  
US = 0x1000
```

They advance by one hexadecimal place:

```
0x0001 << 4 = 0x0010  
0x0010 << 4 = 0x0100  
0x0100 << 4 = 0x1000  
0x1000 << 4 = 0x10000
```

Conceptually:

```
FS <<₄ GS <<₄ RS <<₄ US <<₄ CARRY
```


where:

```
<<4  
means advance by one hexadecimal digit
```

This ladder answers a different question from the four-bit scope mask.

```
scope4 bits  
identify which scopes are visible  
  
place-value masks  
identify where each gauge stands in the larger addressed word
```

Both can coexist.

7. LOGOS, NOMOS, and PATHOS

The four scope bits are protected by three derived Hamming relations:

```
LOGOS  
NOMOS  
PATHOS
```

The canonical Hamming arrangement is:

```
position 1 = LOGOS  
position 2 = NOMOS  
position 3 = FS  
position 4 = PATHOS  
position 5 = GS  
position 6 = RS  
position 7 = US
```

Compactly:

```
[L N F P G R U]
```

The parity equations are:

LOGOS
= FS XOR GS XOR US

NOMOS
= FS XOR RS XOR US

PATHOS
= GS XOR RS XOR US

Equivalently:

LOGOS XOR FS XOR GS XOR US = 0

NOMOS XOR FS XOR RS XOR US = 0

PATHOS XOR GS XOR RS XOR US = 0

The existing Hamming integration canon explicitly treats FS/GS/RS/US as primary scope data and LOGOS/NOMOS/PATHOS as derived integrity relations.

8. What the Three Integrity Relations Mean

LOGOS

LOGOS checks:

FS
GS
US

It asks:

Does the unit remain structurally coherent
with its file and group relation?

LOGOS concerns coherence.

NOMOS

NOMOS checks:

FS
RS
US

It asks:

Does the unit obey the declared file,
record, and boundary arrangement?

NOMOS concerns law, ordering, and admissibility.

PATHOS

PATHOS checks:

GS
RS
US

It asks:

Has the carried unit remained continuous
through group and record traversal?

PATHOS concerns continuity and carried state.

The concise narrative is:

LOGOS coheres.

NOMOS orders.

PATHOS carries.

9. The Hamming Syndrome

When a codeword is received, the three integrity equations are recomputed:

```
sLOGOS
= LOGOS XOR FS XOR GS XOR US

sNOMOS
= NOMOS XOR FS XOR RS XOR US

sPATHOS
= PATHOS XOR GS XOR RS XOR US
```

The syndrome value is:

```
syndrome
=
sLOGOS
+ 2 × sNOMOS
+ 4 × sPATHOS
```

The mapping is:

```
000
no detected single-bit error

001
LOGOS

010
NOMOS

011
FS

100
PATHOS

101
GS

110
RS

111
US
```

This is the Fano-Hamming correspondence: the three syndrome bits identify one of seven nonzero incidence positions.

The syndrome answers:

Which local incidence position appears corrupted?

It does not answer:

When was that state active?

How should it be projected?

Should the relation be semantically accepted?

10. Extended Hamming and the Full Byte

For the full byte carrier, an extended Hamming `[8,4,4]` profile may be used.

The eighth bit is a global parity witness.

The eight-bit carrier provides:

single-error correction

double-error detection

A representative extended codeword surface is:

```
00 0F 33 3C
55 5A 66 69
96 99 A5 AA
C3 CC F0 FF
```

The OMI lattice documents use this code as a finite binary doorway, while explicitly distinguishing the standard code from the larger OMI interpretive ladder.

Implementation rule:

Hamming validity establishes transport integrity.

It does not establish semantic truth.

11. LL, MM, and NN as Delta3 Time

LL, MM, and NN are temporal coordinates:

LL = previous
MM = present
NN = forward

They answer:

At which local temporal position
is this relation being evaluated?

The window is:

LL → MM → NN

The rolling law is:

(LL, MM, NN)
→
(MM, NN, NEW)

Thus:

old MM becomes new LL

old NN becomes new MM

NEW becomes new NN

The positions remain fixed.

The values move through them.

12. A Hamming Cell at Every Delta Position

Each temporal position carries its own complete Hamming epistemic cell.

```
Cell_LL
=
Hamming(scope_LL)

Cell_MM
=
Hamming(scope_MM)

Cell_NN
=
Hamming(scope_NN)
```

Each cell contains:

```
scope4

derived LOGOS/NOMOS/PATHOS bits

seven-bit codeword

syndrome

correction status

validation status
```

The full Delta3 object is therefore:

```
three Hamming cells across time
```

not:

```
one enlarged Hamming word
```

This distinction is essential.

13. The Canonical Temporal Routing

The natural routed correspondence is:

```
LL → LOGOS  
MM → NOMOS  
NN → PATHOS
```

This means:

```
LL / LOGOS  
previous relation checked for inherited coherence  
  
MM / NOMOS  
present relation checked for current law and boundary  
  
NN / PATHOS  
forward relation checked for continuity
```

The narrative becomes:

```
LOGOS remembers.  
  
NOMOS resolves.  
  
PATHOS carries.
```

But this routed correspondence does not replace the complete Hamming checks.

Every temporal cell may still be checked by all three integrity relations.

14. The Full Temporal-Integrity Matrix

The full evaluation is a `3 × 3` matrix:

	LOGOS	NOMOS	PATHOS
LL previous	a	b	c
MM present	d	e	f


```
NN forward      g      h      i
```

Rows represent:

```
temporal positions
```

Columns represent:

```
integrity dimensions
```

Formally:

```
TemporalIntegrity[delta][axis]
```

The routed correspondence forms the matrix diagonal:

```
LL / LOGOS  
  
MM / NOMOS  
  
NN / PATHOS
```

This diagonal may be published as a compact summary witness.

The complete matrix should remain available for audit and debugging.

15. Delta3 Axis Witness

The compact diagonal can be packed as three bits:

```
bit 0  
LL/LOGOS failure  
  
bit 1  
MM/NOMOS failure
```

bit 2
NN/PATHOS failure

Interpretation:

000
all routed checks pass

001
previous coherence failure

010
present law or boundary failure

100
forward continuity failure

Combined values indicate multiple routed failures.

This value must not be called a Hamming syndrome.

Use:

Delta3AxisWitness

because its bit meanings are temporal-integrity routes.

16. Tetragrammatron Relation Governor

After local transport integrity and temporal placement are resolved, the Tetragrammatron Relation Governor adjudicates relation closure.

Its witness family is:

0x1E
0x3C
0x78

The Polybius nibble field is:

```
0 1 2 3
4 5 6 7
8 9 A B
C D E F
```

The principal diagonals are:

```
D+ = {0,5,A,F}
D- = {3,6,9,C}
```

Their XOR closures are:

```
0 ^ 5 ^ A ^ F = 0
3 ^ 6 ^ 9 ^ C = 0
```

Their sums are:

```
SUM(D+) = 0x1E
SUM(D-) = 0x1E
```

The centered fold is:

```
0x1E + 0x1E = 0x3C
```

The shifted/full-system fold is:

```
0x3C << 1 = 0x78
```

Therefore:

```
0x1E
relation closure

0x3C
centered fold
```

0x78
shifted/full-system fold

Tetragrammatron answers:

Does the resolved temporal relation close lawfully?

It does not answer where the relation occurred or how it should be projected.

17. Metatron Incidence Scribe

Metatron records the incidence path.

Its witness family is:

0x18

0x0001
0x0010
0x0100
0x1000
0x10000

The four gauges under open/sealed polarity create:

4 gauges × 2 polarities = 8 points

The six gauge pairs are:

FS-GS
FS-RS
FS-US
GS-RS
GS-US
RS-US

The incidence balance is:

8 points × degree 3 = 24

6 circles × degree 4 = 24

Thus:

24 decimal = 0x18 = 4!

Metatron records:

which gauge was active

whether it was open or sealed

which gauge-pair plane was traversed

which hexadecimal place was selected

whether carry reached 0x10000

Metatron answers:

Where and under what incidence
did the relation occur?

It does not determine Polybius closure.

18. Gnomonic Projection Azimuth

The Gnomonic Projection Azimuth orients the accepted or inspectable temporal relation.

Its byte carrier is:

0x00..0xFF

This provides 256 discrete circular positions.

A conventional angular mapping is:

```
angle(a)
=
a × 360° / 256
```

Canonical quarter positions are:

```
0x00 = 0°
0x40 = 90°
0x80 = 180°
0xC0 = 270°
```

After:

```
0xFF
```

the circle wraps to:

```
0x00
```

The azimuth answers:

```
How is the checked temporal relation oriented
from this observer-relative projection axis?
```

19. The Balanced Projection Words

The two balanced byte phases are:

```
0x55 = 01010101
0xAA = 10101010
```

They satisfy:

```
0x55 XOR 0xAA = 0xFF
```

Each contains:

four ones
four zeros

Their sixteen-bit combinations are:

0x55AA
0xAA55

Each contains:

eight ones
eight zeros

and:

$0x55AA \text{ XOR } 0xAA55 = 0xFFFF$

These are complementary projection phases.

They are not:

Hamming syndromes
Delta3 positions
relation-validation witnesses
incidence witnesses

They belong to the Gnomonic Projection Azimuth because they preserve alternating contrast and complete complementarity.

20. Hamming and Azimuth Together

The full byte azimuth is the ambient eight-bit carrier:

$$B^8 = \{0,1\}^8$$

The extended Hamming code selects sixteen certified codewords from the 256 possible byte states.

Both:

```
0x55  
0xAA
```

are representative extended-Hamming codewords.

Therefore the projection layer may distinguish:

```
raw azimuth  
any byte position from 0x00..0xFF  
  
certified azimuth  
an azimuth position that is also a valid extended Hamming codeword
```

This gives a useful implementation status:

```
AZIMUTH_RAW  
  
AZIMUTH_HAMMING_CERTIFIED  
  
AZIMUTH_CORRECTED  
  
AZIMUTH_UNRESOLVED
```

But Hamming certification still means only:

```
the projection word satisfies the selected code
```

It does not mean:

```
the projected meaning is true
```


21. Circular Distance and Hamming Distance

The azimuth layer should preserve two distances.

Circular displacement

```
circular_delta(a,b)
=
(b - a) mod 256
```

This measures:

```
how far the cursor rotated
```

Hamming displacement

```
hamming_delta(a,b)
=
popcount(a XOR b)
```

This measures:

```
how many binary orientation components changed
```

For:

```
a = 0x55
b = 0xAA
```

the Hamming distance is:

```
popcount(0xFF) = 8
```

which is the maximum byte distance.

The circular distance is a separate value.

This distinction is useful because two azimuth positions can be numerically close but structurally different, or numerically distant but structurally similar.

22. The Combined Runtime Coordinate

A complete relation state should not be represented by one overloaded integer.

It should contain distinct coordinates.

```
scope
integrity
time
incidence
azimuth
validation
```

A conceptual coordinate is:

```
RelationState =
(
    ScopeCoordinate,
    HammingIntegrity,
    DeltaPosition,
    IncidenceRecord,
    Azimuth,
    RelationVerdict
)
```

In compact notation:

```
R =
(scope4, syndrome3, delta, incidence, azimuth, verdict)
```

Each coordinate keeps its provenance.

23. Recommended C Data Model

```
#include <stdbool.h>
#include <stdint.h>
```

```

typedef enum {
    DELTA_LL = 0,
    DELTA_MM = 1,
    DELTA_NN = 2
} DeltaPosition;

typedef enum {
    AXIS_LOGOS = 0,
    AXIS_NOMOS = 1,
    AXIS_PATHOS = 2
} IntegrityAxis;

typedef enum {
    PLANE_LOW_CONTROL = 0,
    PLANE_LOW_AFFINE,
    PLANE_HIGH_CONTROL,
    PLANE_HIGH_PROJECTIVE
} OmnicronPlane;

typedef enum {
    POLARITY_SEALED = 0,
    POLARITY_OPEN = 1
} GaugePolarity;

typedef enum {
    CELL_CLEAN = 0,
    CELL_CORRECTED,
    CELL_DOUBLE_ERROR,
    CELL_UNRESOLVED
} CellStatus;

typedef struct {
    bool fs;
    bool gs;
    bool rs;
    bool us;
} Scope4;

typedef struct {
    bool logos;
    bool nomos;
    bool pathos;
} Integrity3;

typedef struct {
    Scope4 data;
    Integrity3 check;

```

```

    uint8_t codeword7;
    uint8_t syndrome3;
    uint8_t corrected_codeword7;

    CellStatus status;
} HammingEpistemicCell;

typedef struct {
    DeltaPosition position;
    HammingEpistemicCell cell;
} TemporalCell;

typedef struct {
    TemporalCell ll;
    TemporalCell mm;
    TemporalCell nn;

    uint8_t diagonal_witness;
} Delta3Window;

typedef struct {
    uint8_t absolute_byte;
    uint8_t local_position;
    OmnicronPlane plane;
} OmnicronCursor;

typedef struct {
    uint16_t fs;
    uint16_t gs;
    uint16_t rs;
    uint16_t us;
    uint32_t carry;

    GaugePolarity polarity;
    uint8_t incidence_witness;
} MetatronIncidenceRecord;

typedef struct {
    uint8_t azimuth;
    uint16_t orientation_word;

    uint8_t hamming_distance;
    uint8_t circular_displacement;

    bool hamming_certified;
} GnomonicProjection;

```

```

typedef struct {
    uint8_t relation_witness;
    uint8_t centered_fold;
    uint8_t shifted_fold;

    bool diagonal_xor_valid;
    bool relation_valid;
} TetragrammatronVerdict;

typedef struct {
    OmnicronCursor cursor;
    Delta3Window delta;
    MetatronIncidenceRecord incidence;
    TetragrammatronVerdict relation;
    GnomonicProjection projection;

    bool accepted;
} OmiRelationState;

```

24. Encoding a Hamming Epistemic Cell

The check bits are derived from the scope bits:

```

static Integrity3 derive_integrity(Scope4 s) {
    Integrity3 i;

    i.logos = s.fs ^ s.gs ^ s.us;
    i.nomos = s.fs ^ s.rs ^ s.us;
    i.pathos = s.gs ^ s.rs ^ s.us;

    return i;
}

```

Pack the canonical Hamming order:

```

bit 0 = LOGOS
bit 1 = NOMOS
bit 2 = FS
bit 3 = PATHOS
bit 4 = GS

```

```
bit 5 = RS
bit 6 = US
```

```
static uint8_t pack_codeword7(
    Scope4 s,
    Integrity3 i
) {
    return (uint8_t)(
        ((uint8_t)i.logos << 0) |
        ((uint8_t)i.nomos << 1) |
        ((uint8_t)s.fs << 2) |
        ((uint8_t)i.pathos << 3) |
        ((uint8_t)s.gs << 4) |
        ((uint8_t)s.rs << 5) |
        ((uint8_t)s.us << 6)
    );
}
```

25. Computing the Syndrome

```
static uint8_t syndrome7(uint8_t codeword) {
    uint8_t logos = (codeword >> 0) & 1u;
    uint8_t nomos = (codeword >> 1) & 1u;
    uint8_t fs = (codeword >> 2) & 1u;
    uint8_t pathos = (codeword >> 3) & 1u;
    uint8_t gs = (codeword >> 4) & 1u;
    uint8_t rs = (codeword >> 5) & 1u;
    uint8_t us = (codeword >> 6) & 1u;

    uint8_t s_logos =
        logos ^ fs ^ gs ^ us;

    uint8_t s_nomos =
        nomos ^ fs ^ rs ^ us;

    uint8_t s_pathos =
        pathos ^ gs ^ rs ^ us;

    return (uint8_t)(
        s_logos |
        (s_nomos << 1) |
        (s_pathos << 2)
    );
}
```

```

    );
}

```

For a presumed single-bit error:

```

static uint8_t correct_single_error7(
    uint8_t codeword,
    uint8_t syndrome
) {
    if (syndrome == 0u) {
        return codeword;
    }

    return (uint8_t)(
        codeword ^ (1u << (syndrome - 1u))
    );
}

```

A frame-level checksum, receipt hash, or [8,4,4] extended parity profile is still needed to safely detect multi-bit corruption.

26. Rolling the Delta3 Window

```

static void delta3_advance(
    Delta3Window *window,
    HammingEpistemicCell next
) {
    window->ll = window->mm;
    window->ll.position = DELTA_LL;

    window->mm = window->nn;
    window->mm.position = DELTA_MM;

    window->nn.position = DELTA_NN;
    window->nn.cell = next;
}

```

The values move.

The positional roles are reassigned.

This means a cell may be evaluated:

```
first as NN/PATHOS  
  
later as MM/NOMOS  
  
finally as LL/LOGOS
```

That is the temporal lifecycle of one relation candidate.

27. Extracting the Delta3 Diagonal

```
static uint8_t delta3_axis_witness(  
    const Delta3Window *window  
) {  
    uint8_t ll_logos =  
        (window->ll.cell.syndrome3 >> 0) & 1u;  
  
    uint8_t mm_nomos =  
        (window->mm.cell.syndrome3 >> 1) & 1u;  
  
    uint8_t nn_pathos =  
        (window->nn.cell.syndrome3 >> 2) & 1u;  
  
    return (uint8_t)(  
        ll_logos |  
        (mm_nomos << 1) |  
        (nn_pathos << 2)  
    );  
}
```

This compact value summarizes:

```
previous coherence  
  
present law  
  
forward continuity
```

It does not replace the three complete syndromes.

28. Resolving the Omnicron Plane

```
static OmnicronPlane resolve_plane(uint8_t byte) {
    if (byte <= 0x1Fu) {
        return PLANE_LOW_CONTROL;
    }

    if (byte <= 0x7Fu) {
        return PLANE_LOW_AFFINE;
    }

    if (byte <= 0x9Fu) {
        return PLANE_HIGH_CONTROL;
    }

    return PLANE_HIGH_PROJECTIVE;
}
```

The local position must be calculated relative to the active plane:

```
static uint8_t plane_local(
    uint8_t byte,
    OmnicronPlane plane
) {
    switch (plane) {
        case PLANE_LOW_CONTROL:
            return byte;

        case PLANE_LOW_AFFINE:
            return (uint8_t)(byte - 0x20u);

        case PLANE_HIGH_CONTROL:
            return (uint8_t)(byte - 0x80u);

        case PLANE_HIGH_PROJECTIVE:
            return (uint8_t)(byte - 0xA0u);
    }

    return 0u;
}
```

This is better than treating the runtime as one undifferentiated 256-entry array.

29. Resolving Metatron Incidence

```
static MetatronIncidenceRecord
resolve_incidence(GaugePolarity polarity) {
    MetatronIncidenceRecord r;

    r.fs = 0x0001u;
    r.gs = (uint16_t)(r.fs << 4);
    r.rs = (uint16_t)(r.gs << 4);
    r.us = (uint16_t)(r.rs << 4);

    r.carry = ((uint32_t)r.us << 4);

    r.polarity = polarity;
    r.incidence_witness = 0x18u;

    return r;
}
```

A complete implementation should additionally record:

```
active gauge
active gauge pair
open/sealed point
circle membership
place-value lane
carry state
```

30. Resolving the Gnomonic Projection

```
static uint8_t circular_displacement8(
    uint8_t from,
    uint8_t to
) {
```

```
    return (uint8_t)(to - from);  
}
```

```
static uint8_t popcount8(uint8_t x) {  
    uint8_t count = 0;  
  
    while (x != 0u) {  
        count += (uint8_t)(x & 1u);  
        x >>= 1;  
    }  
  
    return count;  
}
```

```
static uint8_t hamming_distance8(  
    uint8_t a,  
    uint8_t b  
) {  
    return popcount8((uint8_t)(a ^ b));  
}
```

A projection record may be constructed as:

```
static GnomonicProjection project_azimuth(  
    uint8_t previous,  
    uint8_t current,  
    bool forward_phase  
) {  
    GnomonicProjection p;  
  
    p.azimuth = current;  
  
    p.orientation_word =  
        forward_phase ? 0xAA55u : 0x55AAu;  
  
    p.circular_displacement =  
        circular_displacement8(previous, current);  
  
    p.hamming_distance =  
        hamming_distance8(previous, current);  
  
    p.hamming_certified = false;  
}
```

```
    return p;
}
```

The certification flag should be set only after the byte has been tested against the selected extended Hamming code.

31. Tetragrammatron Validation

A minimal deterministic relation check may store the known witnesses:

```
static TetragrammatronVerdict
validate_relation(void) {
    TetragrammatronVerdict v;

    uint8_t d_plus_xor =
        0x0u ^ 0x5u ^ 0xAu ^ 0xFu;

    uint8_t d_minus_xor =
        0x3u ^ 0x6u ^ 0x9u ^ 0xCu;

    v.relation_witness = 0x1Eu;
    v.centered_fold = 0x3Cu;
    v.shifted_fold = 0x78u;

    v.diagonal_xor_valid =
        d_plus_xor == 0u &&
        d_minus_xor == 0u;

    v.relation_valid =
        v.diagonal_xor_valid &&
        ((uint8_t)(v.relation_witness * 2u)
         == v.centered_fold) &&
        ((uint8_t)(v.centered_fold << 1)
         == v.shifted_fold);

    return v;
}
```

In the real runtime, the relation values must be derived from the candidate being adjudicated rather than merely loading canonical constants.

32. The Acceptance Rule

A complete acceptance policy may require:

```
local Hamming integrity passes  
  
or a permitted single error is corrected  
  
no unresolved double error exists  
  
Delta3 temporal routing is well formed  
  
Omicron plane transition is lawful  
  
Omicron position is reachable  
  
Tetragrammatron relation closure passes  
  
Metatron incidence record is complete  
  
projection metadata is internally consistent
```

Conceptually:

```
accepted =  
    transport_integrity  
AND temporal_integrity  
AND lawful_reachability  
AND relation_closure  
AND complete_incidence_record
```

Projection should normally not determine acceptance.

It is metadata derived after or alongside adjudication.

33. Receipt Structure

A receipt should preserve each layer separately.

```
{  
  "scope": {
```

```

    "fs": true,
    "gs": true,
    "rs": false,
    "us": true
  },
  "integrity": {
    "codeword": "0x57",
    "syndrome": "0x00",
    "status": "clean"
  },
  "delta": {
    "position": "MM",
    "axis": "NOMOS",
    "diagonal_witness": "0x00"
  },
  "omicron": {
    "byte": "0x4F",
    "sector": 2,
    "local": 15
  },
  "omnicron": {
    "plane": "LOW_AFFINE",
    "local_position": 47
  },
  "incidence": {
    "witness": "0x18",
    "polarity": "open",
    "gauge": "RS",
    "carry": "0x10000"
  },
  "relation": {
    "witness": "0x1E",
    "centered_fold": "0x3C",
    "shifted_fold": "0x78",
    "valid": true
  },
  "projection": {
    "azimuth": "0xAA",
    "orientation_word": "0xAA55",
    "circular_displacement": 85,
    "hamming_distance": 8
  },
  "accepted": true
}

```

The exact serialization format may differ, but the provenance separation should remain.

34. The Complete Execution Pipeline

1. RECEIVE

Receive a byte stream or local declaration.

2. DEFRAME

Apply COBS decoding if the transport uses COBS framing.

3. UNPACK

Recover Hamming [7,4,3] or [8,4,4] code units.

4. CHECK LOCAL INTEGRITY

Derive or recompute LOGOS/NOMOS/PATHOS.

5. CORRECT LOCALLY

Correct one local bit where the chosen profile permits it.

6. RECOVER SCOPE

Recover FS/GS/RS/US.

7. RESOLVE OMICRON

Determine the bounded low gauge position and reachable sign.

8. RESOLVE OMNICRON

Determine the active low/high and control/space plane.

9. PLACE IN DELTA3

Insert the relation as LL, MM, or NN.

10. ROUTE TEMPORAL INTEGRITY

Evaluate LL/LOGOS, MM/NOMOS, and NN/PATHOS.

11. RESOLVE INCIDENCE

Determine gauge, pairwise plane, open/sealed polarity, hexadecimal place, and carry.

12. ADJUDICATE RELATION

Apply the Tetragrammatron closure and fold laws.

13. DECIDE

Accept, reject, or retain as unresolved.

14. RECEIPT

Record every witness with its provenance.

15. PROJECT

Resolve the Gnomonic Projection Azimuth.

16. CARRY

Serialize and transport the receipt and optional projection.

35. Why This Architecture Matters

Without these separations, several dangerous collapses become possible.

A corrected bit
could be mistaken for semantic truth.

A present temporal state
could be mistaken for an accepted state.

A projection
could be mistaken for the source relation.

A gauge cursor
could be mistaken for relation authority.

A scribed incidence path
could be mistaken for closure proof.

The architecture prevents this by assigning each question to one component.

Omicron

Where is the bounded sign?

Omicron

Which runtime plane is active?

Hamming cell

Did the local representation survive?

Delta3

When is the relation being considered?

Tetragrammatron

Does the relation close?

Metatron

Where did it cross?

Gnomonic Projection Azimuth

How does it appear from here?

Receipt

What was actually accepted?

36. Canonical Narrative

The relation begins as a bounded scope object.

FS, GS, RS, and US establish its containment.

LOGOS, NOMOS, and PATHOS check whether that containment has survived transmission and traversal.

LL, MM, and NN place successive checked cells into previous, present, and forward positions.

Omicron makes the sign reachable inside the low gauge.

Omicron resolves the sign against the complete runtime plane.

Tetragrammatron determines whether the relation closes.

Metatron records its gauge incidence and carry path.

Receipt records the decision.

The Gnomonic Projection Azimuth orients the recorded relation for observer-relative inspection.

Projection never becomes acceptance.

37. One-Line Canon

FS, GS, RS, and US establish the scoped relation; LOGOS, NOMOS, and PATHOS protect it as a Hamming-coded integrity cell; LL, MM, and NN move successive cells through previous, present, and forward time; Omicron supplies the bounded low gauge; Omnicron resolves the complete circular byte runtime; Tetragrammatron validates relation closure; Metatron scribes gauge incidence and hexadecimal carry; the Gnomonic Projection Azimuth orients the accepted temporal relation for inspection; and receipt preserves the decision without collapsing transport integrity, temporal position, incidence, projection, or semantic acceptance.

38. Final Canon Lock

FS/GS/RS/US

establish where the relation is scoped.

LOGOS/NOMOS/PATHOS

establish whether the scope survived coherently.

LL/MM/NN

establish when the checked relation stands.

Omicron

establishes the bounded sign-and-place position.

Omnicon

establishes the active whole-runtime plane.

Tetragrammatron

establishes whether the relation closes.

Metatron

records how the relation crossed the gauge field.

Gnomonic Projection Azimuth
establishes how the temporal relation appears
from the observer-relative axis.

Receipt
records what was accepted.

The Hamming syndrome says what was corrupted.

Delta3 says when it was present.

Incidence says where it crossed.

Azimuth says how it appears.

Closure says whether it holds.

Receipt says what the system accepted.